Mining of Massive Datasets, ch. 3

"Similar strings → similar hashes" ~ $O(N)$

$$J(C_1, C_2) = \frac{C_1 \cap C_2}{C_1 \cup C_2}$$

Steps: shingling (sparse) → Min-hashing to short signatures → LSH to generate candidate pairs

"k-shingles" (n-grams) → words or letters. Here, letters.
JOHN 2-grams: {JO, OH, HN}
optional: hash these again to improve storage {2, 1, 7}

Represent every string by its k-shingles

$$\text{Jaccard}(\text{JOHN}, \text{JOHNN}) = \frac{|\text{JO, OH, HN}|}{|\text{JO, OH, HN, NN}|} = \frac{3}{4}$$

For short strings, you can pick small k. ≈ 5
For large ones, larger k so you don't have all shingles in every string.

Now: $\frac{N(N-1)}{2}$ comparisons ... still. Can't do pairwise, even on one-hot encoded vectors

One-hot encoding:

	JOHN	JOHNN
J	1	1
O	1	1
H	1	1
N	1	1
E	0	0

With many unique shingles, this matrix is very sparse and requires a lot of memory to store.

We want to find similar columns ~~without~~ while computing small signatures.

Idea: "hash" each column (string's one-hot encoding) so that the hashes all fit in memory.

Bonus: $\widehat{J}(C_1, C_2) \propto Pr(h(c_1) = h(c_2))$

For Jaccard similarity, this hash function is called the "min-hash"

Min-hashing

Let's randomly permute the rows in the input matrix.

N unique shingles × N docs

	S_1	S_2	S_3	S_4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

Permutation		
4	5	1
5	1	2
2	4	5
1	3	4
3	2	3
Φ_1	Φ_2	Φ_3

The "min" part is what's the min col number before we see a 1?

Signature			
Φ_1	1	2	1	1
Φ_2	3	4	1	3
Φ_3	1	5	2	1

Note the property here:

For a random permutation Φ_i, $Pr(\Phi_i(S_1) = \Phi_i(S_2)) = \text{Jaccard}(S_1, S_2)$

Why? Because if for a random row, ~~they are the same, then iterate to next row and determine~~

$(1,1) \Rightarrow x$

$(0,1)$ or $(1,0) \Rightarrow y$

$(0,0) \Rightarrow$ ignore and move to next row

$\Rightarrow \frac{x}{x+y} \Rightarrow Pr(\Phi_i(S_1) = \Phi_i(S_2))$

This is also the Jaccard similarity!

Now, Similarity of signatures is fraction of permutations in which they agree.

Expected fraction as N permutations $\to \infty$ is equal to Jaccard.

So now we have a much smaller representation, an $N_{hashes} \times M_{docs}$ dense matrix. $N_{hashes} = 100$ is common. permutations

BUT: permutations are really memory-intensive. And then you have to store the whole matrix in memory. one-hot

So instead we are going to make two tweaks to this approach

Tweak 2: only do the first M rows

Tweak 1: Instead of N permutations, let's write N functions that hash (assign) row numbers to buckets, where $N_{buckets} \ll N_{shingles}$.

How do we pick them?
Let's use universal hashing
$$h(x) = ((ax + b) \bmod p) \bmod N$$
where a, b are random integers and p is a prime $> N$

For simplicity, let's say
$h_1 = (x + 1) \bmod 5$ and $h_2 = (3x + 1) \bmod 5$
where x is the row number.

As long as the number of buckets is large and we don't have too many collisions, this will suffice as an approximation of random row permutations.

Then I don't need to actually permute, and I can store only S_i at any given time (i.e., I can work string by string.)

So, now, to get the signature matrix

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

initialize to ∞.

For each string S_i,
  For each hash function, ~~compute hash~~
    compute hash of each entry.
    In the signature matrix, every entry $sig(h, S_i)$ gets min(current value, hash) for all S_i where $S_i(|h|) = 1$ (not 0)

S_1	S_2	S_3	S_4	h_1	h_2
1	0	0	1	1	1
0	0	1	0	2	4
0	1	0	1	3	2
1	0	1	1	4	0
0	0	1	0	0	3

	S_1	S_2	S_3	S_4
h_1	∞	∞	∞	∞
h_2	∞	∞	∞	∞

~~h_1 (first row)~~ We are going to fill these in with the smallest hash value for which every S_i has a 1 in its row.

~~[illegible]~~

So let's look at first row

S_1 and $S_4 = 1$ so let's put $(h_1, h_2) = (1, 1)$ in the columns for S_1 and S_4.

1	∞	∞	1
1	∞	∞	1

Now row 2, $S_3 = 1$ so put $(h_1, h_2) = (2, 4)$ for column with S_3

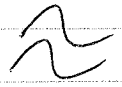
1	∞	2	1
1	∞	4	1

Now row 3, S_2 and $S_4 = 1$ so put $(h_1, h_2) = (3, 2)$ for column with S_2 and S_4 but $1 < 3$ and $1 < 2$ so no updates for S_4

1	3	2	1
1	2	4	1

Proceed to rows 4 and 5.

get: $\begin{bmatrix} 1 & 3 & 0 & 1 \\ 0 & 2 & 0 & 0 \end{bmatrix}$



Tweak 2: This still requires handling a matrix with N rows = N shingles for each minhash function. Instead of this, we can instead approximate this task by examining only M rows for each function. ~~This may not actually get~~ where $M << k$. From this, we actually get k/m reduction in the amount of compute. Now, this is still an approximation. If some $S_i = 0$ $\forall$ first m rows, the function tells us nothing.

But think about two random S_i, S_j.

1. If both $= 0$ $\forall m$, then we just don't learn anything from this function.
2. If only one $= 0$ $\forall k$, then we know they are quite different and only one gets updated (great!)

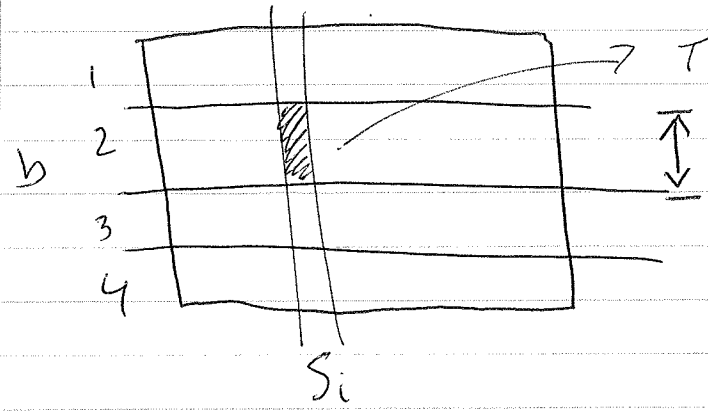
As long as (1) is rare, we don't have a serious problem (and we can actually spend time computing more hash functions!).

Now what? We have $N_{hashes} \times N_{obsv.}$ signature matrix. If that's 250 x 1M, we are ~ 1GB. But still $\binom{1M}{2}$ comparisons (~500B). Not as bad as before, but not great. We could hash them all to buckets ~~where buckets~~ but we would only catch items that are exactly the same signatures!

Locality-sensitive hashing

divides the signature matrix M into b bands of r rows where $b \cdot r = N_{hashes}$.

(N rows of M, or # of minhash functions / signatures for each string).



"Candidate" string pairs have the same signature for at least one band.

How will we find candidate pairs? Hash all signatures to buckets and look at buckets w/ more than one item inside.

Ex: 100,000 strings (M cols) → 40 MB of info!
100 signatures (M rows)

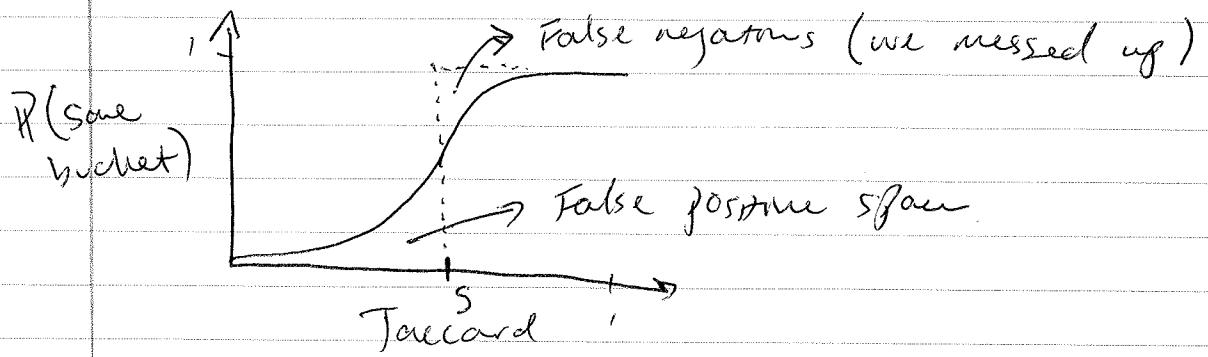
$b = 20$, $r = 5$. Find pairs with Jaccard ≥ 0.8.

$P(C_1$ & C_2 in same band) $= (0.8)^5$ (all five entries in band have to be the same)

$P($ not similar in ANY band$) = \left(1 - (0.8)^5\right)^{20} \approx 0.00035$

Interpretation: We miss 1 of every 3000 pair comparisons when Jaccard ≥ 0.8

Function is $1-(1-s^r)^b$



Before "clustering" for blocking, we could go through and drop all the false positives.

~~Salting~~ ✓

~~Zoomer-join~~ ✓

~~custom levels~~ ✓

For dupmdb

~~blocking~~ rules to generate predictions
take a "salting - partitions" argument.

Threads take 122,880 rows unless you change the configuration.

So with 1M input rows, Thats

$1M / 122880 \rightsquigarrow 9$ threads (base parallelism)

Then ~~Blocking rules x 3~~

~~Rules x Salts~~ $\sum_{\text{blocks}} N$ salts

So if I have three blocking nodes and one of them has 2 splitting partitions

$1 + 1 + 2 = 4$ blocking partitions

Then,

base parallelism $\times 4 =$ overall parallelism.

For Spark

Here, the size of the blocks matters.